GPE Go API - Documento de Diseño

Fecha: 2026-02-02 **Proyecto:** API REST para promoción de turismo y comercios de Guadalupe, N.L.
Contexto: Mundial de Fútbol 2026

---

1. Descripción General

API REST pública que será consumida por:

- Aplicación móvil
- Sitio web

Propósito: Promover el turismo y los comercios locales del municipio de Guadalupe, Nuevo León, aprovechando el Mundial de Fútbol 2026.

---

2. Arquitectura de Archivos

```
/gpe_go_api/
│
├── .env                        # Variables de entorno (no en git)
├── .env.example                # Plantilla de variables
├── .gitignore                  # Excluir .env
│
├── app_config.php              # Configuración y conexión BD
├── bouncer.php                 # Seguridad: JWT y permisos
├── inputs.php                  # Gateway: validación y anti-SQL injection
│
├── funciones/
│   ├── index.php               # Helpers: responder(), encriptar(), desencriptar()
│   ├── funciones_usuarios.php
│   ├── funciones_categorias.php
│   ├── funciones_lugares.php
│   ├── funciones_eventos.php
│   ├── funciones_favoritos.php
│   └── funciones_resenas.php
│
├── inputs/
│   ├── index.php
│   ├── inputs_usuarios.php
│   ├── inputs_categorias.php
│   ├── inputs_lugares.php
│   ├── inputs_eventos.php
│   ├── inputs_favoritos.php
│   └── inputs_resenas.php
│
└── docs/
    └── plans/
        └── (este archivo)
```

---

3. Decisiones Técnicas

Aspecto	Decisión
Lenguaje	PHP
Base de datos	MySQL
Autenticación	JWT + 2FA por email
Código 2FA	6 dígitos, expira en 4 horas
Config local	Archivo .env
Config producción	AWS Parameter Store (futuro)
Borrado	Lógico (campo `enabled` DEFAULT 1)
Email	Encriptado AES-256 (IV fijo para búsqueda)
Código 2FA	Encriptado AES-256
Protección	Sanitización + detección SQL injection
Formato respuestas	JSON: `success`, `data`, `message`

---

4. Roles y Permisos

Rol	Capacidades
`publico`	Lectura + favoritos, reseñas, ratings
`comercio`	+ Registrar su comercio (requiere aprobación)
`moderador`	+ Gestionar lugares, eventos, reseñas
`admin`	+ Gestionar usuarios y configuración del sistema

---

5. Esquema de Base de Datos

5.1 Convenciones

- Todas las tablas usan prefijo `tb_`
- Todas las tablas tienen campo `id` (INT AUTO_INCREMENT PRIMARY KEY)
- Todas las tablas tienen campo `enabled` (TINYINT DEFAULT 1) para borrado lógico

5.2 Tablas

tb_usuarios

```
CREATE TABLE tb_usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
```

```sql
    password VARCHAR(255),
    codigo_expira DATETIME,
    rol ENUM('publico', 'comercio', 'moderador', 'admin') DEFAULT 'publico',
    enabled TINYINT DEFAULT 1
);
```

tb_categorias

```sql
CREATE TABLE tb_categorias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    enabled TINYINT DEFAULT 1
);
```

tb_lugares

```sql
CREATE TABLE tb_lugares (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    direccion VARCHAR(255),
    telefono VARCHAR(20),
    id_categoria INT NOT NULL,
    id_usuario INT NOT NULL,
    estado ENUM('pendiente', 'aprobado', 'rechazado') DEFAULT 'pendiente',
    enabled TINYINT DEFAULT 1,
    FOREIGN KEY (id_categoria) REFERENCES tb_categorias(id),
    FOREIGN KEY (id_usuario) REFERENCES tb_usuarios(id)
);
```

tb_eventos

```sql
CREATE TABLE tb_eventos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    descripcion TEXT,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,
    tipo ENUM('evento', 'noticia') DEFAULT 'evento',
    id_lugar INT,
    enabled TINYINT DEFAULT 1,
    FOREIGN KEY (id_lugar) REFERENCES tb_lugares(id)
);
```

tb_favoritos

```sql
CREATE TABLE tb_favoritos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
```

```sql
    id_lugar INT,
    id_evento INT,
    enabled TINYINT DEFAULT 1,
    FOREIGN KEY (id_usuario) REFERENCES tb_usuarios(id),
    FOREIGN KEY (id_lugar) REFERENCES tb_lugares(id),
    FOREIGN KEY (id_evento) REFERENCES tb_eventos(id)
);
```

tb_resenas

```sql
CREATE TABLE tb_resenas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    id_lugar INT NOT NULL,
    comentario TEXT,
    calificacion TINYINT NOT NULL,
    enabled TINYINT DEFAULT 1,
    FOREIGN KEY (id_usuario) REFERENCES tb_usuarios(id),
    FOREIGN KEY (id_lugar) REFERENCES tb_lugares(id)
);
```

5.3 Relaciones

```
tb_usuarios (1) ─── (N) tb_lugares      [dueño del comercio]
tb_usuarios (1) ─── (N) tb_favoritos    [usuario guarda favoritos]
tb_usuarios (1) ─── (N) tb_resenas      [usuario deja reseñas]
tb_categorias (1) ── (N) tb_lugares     [categoría del lugar]
tb_lugares (1) ──── (N) tb_eventos      [evento en un lugar, opcional]
tb_lugares (1) ──── (N) tb_resenas      [reseñas del lugar]
```

---

6. Flujo de Request

```
Request HTTP
     │
     ▼
┌──────────────────────────────┐
│  inputs.php (GATEWAY)        │
│  - Valida parámetros requeridos │
│  - Sanitiza datos de entrada  │
│  - Detecta inyección SQL      │
│  - Rutea al input correcto    │
└──────────────────────────────┘
     │
     ▼
┌──────────────────────────────┐
│  bouncer.php                 │
│  - Valida JWT                │
│  - Verifica rol              │
└──────────────────────────────┘
```

```
     │
     ▼
┌──────────────────────────────┐
│  inputs/inputs_*.php         │
│  - Lógica específica del módulo │
│  - Llama a funciones         │
└──────────────────────────────┘
     │
     ▼
┌──────────────────────────────┐
│  funciones/funciones_*.php   │
│  - Operaciones CRUD          │
│  - Consultas a BD            │
└──────────────────────────────┘
     │
     ▼
┌──────────────────────────────┐
│  app_config.php              │
│  - Conexión BD (PDO)         │
│  - Constantes globales       │
└──────────────────────────────┘
```

---

7. Autenticación 2FA

7.1 Flujo

Paso 1: Solicitar código

```
POST /inputs.php?modulo=usuarios&action=solicitar_codigo
Body: { "email": "usuario@ejemplo.com" }

Sistema:
1. Busca usuario por email (encriptado)
2. Genera código aleatorio de 6 dígitos
3. Guarda código encriptado en campo 'password'
4. Guarda codigo_expira = NOW() + 4 horas
5. Envía email con el código
6. Responde: { success: true, message: "Código enviado" }
```

Paso 2: Verificar código

```
POST /inputs.php?modulo=usuarios&action=verificar_codigo
Body: { "email": "usuario@ejemplo.com", "codigo": "482910" }

Sistema:
1. Busca usuario por email
2. Verifica que codigo_expira > NOW()
3. Compara código encriptado
4. Si válido → genera JWT y responde con token
5. Si expiró → error "Código expirado"
6. Si no coincide → error "Código incorrecto"
```

7.2 Estructura JWT (payload)

```
{
  "id": 123,
  "email": "usuario@ejemplo.com",
  "rol": "publico",
  "exp": 1234567890
}
```

---

8. Seguridad

8.1 Encriptación

Email: Encriptación AES-256-CBC con IV fijo (derivado de la key) para permitir búsquedas.

Código 2FA: Encriptación AES-256-CBC.

```
function encriptar_email($email) {
    $key = getenv('ENCRYPTION_KEY');
    $iv = substr(hash('sha256', getenv('ENCRYPTION_KEY')), 0, 16);
    $encrypted = openssl_encrypt(strtolower($email), 'AES-256-CBC', $key, 0, $iv);
    return base64_encode($encrypted);
}

function desencriptar_email($valor) {
    $key = getenv('ENCRYPTION_KEY');
    $iv = substr(hash('sha256', getenv('ENCRYPTION_KEY')), 0, 16);
    return openssl_decrypt(base64_decode($valor), 'AES-256-CBC', $key, 0, $iv);
}
```

8.2 Protección SQL Injection

El gateway `inputs.php` detecta patrones peligrosos:

```
$patrones_peligrosos = [
    '/\bINSERT\b/i',
    '/\bUPDATE\b/i',
    '/\bDELETE\b/i',
    '/\bDROP\b/i',
    '/\bTRUNCATE\b/i',
    '/\bALTER\b/i',
    '/\bCREATE\b/i',
    '/\bEXEC\b/i',
    '/\bUNION\b/i',
    '/\bSELECT\b.*\bFROM\b/i',
    '/--/',
    '/;.*\b(DROP|DELETE|INSERT|UPDATE)\b/i'
];
```

8.3 Bouncer - Funciones

Función	Descripción
`generar_token($usuario)`	Crea JWT con datos del usuario
`validar_token()`	Lee header Authorization, valida JWT, retorna datos o false
`requiere_auth()`	Valida token, si falla retorna error 401
`requiere_rol($roles)`	Valida token + verifica rol, si falla retorna error 403

---

9. Endpoints

9.1 Usuarios

Método	Endpoint	Descripción	Acceso
POST	`?modulo=usuarios&action=registro`	Registrar usuario	Público
POST	`?modulo=usuarios&action=solicitar_codigo`	Solicitar código 2FA	Público
POST	`?modulo=usuarios&action=verificar_codigo`	Verificar código y obtener JWT	Público
GET	`?modulo=usuarios&action=perfil`	Ver perfil propio	Autenticado
PUT	`?modulo=usuarios&action=editar`	Editar perfil propio	Autenticado
GET	`?modulo=usuarios&action=listar`	Listar usuarios	Admin
PUT	`?modulo=usuarios&action=cambiar_rol`	Cambiar rol de usuario	Admin

9.2 Categorías

Método	Endpoint	Descripción	Acceso
GET	`?modulo=categorias&action=listar`	Listar categorías	Público
POST	`?modulo=categorias&action=crear`	Crear categoría	Admin
PUT	`?modulo=categorias&action=editar&id=X`	Editar categoría	Admin
DELETE	`?modulo=categorias&action=eliminar&id=X`	Eliminar categoría	Admin

9.3 Lugares

Método	Endpoint	Descripción	Acceso
GET	`?modulo=lugares&action=listar`	Listar lugares aprobados	Público
GET	`?modulo=lugares&action=ver&id=X`	Ver detalle de lugar	Público
POST	`?modulo=lugares&action=registrar`	Registrar comercio	Comercio

PUT	`?modulo=lugares&action=editar&id=X`	Editar mi comercio	Comercio (dueño)
PUT	`?modulo=lugares&action=aprobar&id=X`	Aprobar lugar	Moderador/Admin
PUT	`?modulo=lugares&action=rechazar&id=X`	Rechazar lugar	Moderador/Admin
DELETE	`?modulo=lugares&action=eliminar&id=X`	Eliminar lugar	Moderador/Admin

9.4 Eventos

Método	Endpoint	Descripción	Acceso
GET	`?modulo=eventos&action=listar`	Listar eventos/noticias	Público
GET	`?modulo=eventos&action=ver&id=X`	Ver detalle de evento	Público
POST	`?modulo=eventos&action=crear`	Crear evento/noticia	Moderador/Admin
PUT	`?modulo=eventos&action=editar&id=X`	Editar evento	Moderador/Admin
DELETE	`?modulo=eventos&action=eliminar&id=X`	Eliminar evento	Moderador/Admin

9.5 Favoritos

Método	Endpoint	Descripción	Acceso
GET	`?modulo=favoritos&action=listar`	Listar mis favoritos	Autenticado
POST	`?modulo=favoritos&action=agregar`	Agregar a favoritos	Autenticado
DELETE	`?modulo=favoritos&action=quitar&id=X`	Quitar de favoritos	Autenticado

9.6 Reseñas

Método	Endpoint	Descripción	Acceso
GET	`?modulo=resenas&action=listar&id_lugar=X`	Listar reseñas de un lugar	Público
POST	`?modulo=resenas&action=crear`	Crear reseña	Autenticado
DELETE	`?modulo=resenas&action=eliminar&id=X`	Eliminar reseña	Moderador/Admin

---

10. Formato de Respuestas

10.1 Respuesta exitosa

```
{
  "success": true,
  "data": { ... },
  "message": "Operación realizada correctamente"
}
```

10.2 Respuesta con listado

```json
{
  "success": true,
  "data": [
    { "id": 1, "nombre": "..." },
    { "id": 2, "nombre": "..." }
  ],
  "total": 25,
  "pagina": 1,
  "por_pagina": 10
}
```

10.3 Respuesta de error

```json
{
  "success": false,
  "error": {
    "codigo": "AUTH_INVALID_TOKEN",
    "mensaje": "El token ha expirado"
  }
}
```

10.4 Códigos HTTP

Código	Uso
200	Operación exitosa
201	Recurso creado exitosamente
400	Error de validación / datos inválidos
401	No autenticado / token inválido
403	Sin permisos para esta acción
404	Recurso no encontrado
500	Error interno del servidor

---

11. Configuración

11.1 Variables de entorno (.env)

```
APP_ENV=development

DB_HOST=localhost
DB_NAME=gpe_go_db
DB_USER=root
```

```
DB_PASS=

JWT_SECRET=tu_clave_secreta_aqui
JWT_EXPIRATION=86400

ENCRYPTION_KEY=tu_clave_de_32_caracteres_aqui
```

11.2 Entornos

- **development:** Lee configuración de archivo `.env`
- **production:** Lee configuración de AWS Parameter Store (JSON)

---

12. Notas de Implementación

1. Todas las consultas SELECT deben filtrar por `WHERE enabled = 1`
2. El borrado es lógico: `UPDATE ... SET enabled = 0`
3. Los emails se normalizan a minúsculas antes de encriptar
4. La calificación en reseñas es de 1 a 5 (TINYINT)
5. Los favoritos pueden ser de lugar O evento, no ambos
6. Las reseñas se publican inmediatamente, moderadores pueden eliminar después